PV
- 1 生产者 1 消费者 1 缓冲区
- 1 生产者 1 消费者 N 缓冲区
- N 生产者 N 消费者 N 缓冲区
- 苹果橘子问题
- 读者写者问题
  - 读者优先
  - 写者优先
  - 平等
- 睡眠的理发师问题
- 农夫猎人问题
- 银行业务问题
- 缓冲区管理
- 售票问题
- 吸烟者问题
- 哲学家就餐问题
  - 有死锁
  - 避免死锁方法
  - 控制人数
  - 分左右手
- 独木桥问题
  - 一方全部通行
  - 桥上最多 K 辆车
  - 每三辆车交替
  - 串行

管程
- 实现
- 读者写者问题
- 哲学家就餐问题
- 生产者消费者问题
- 苹果橘子问题

PV

1 生产者 1 消费者 1 缓冲区

```
Int B;
Semaphore sput;          /* 可以使用的空缓冲区数 */
Semaphore sget;          /* 缓冲区内可以使用的产品数 */
sput = 1;                /* 缓冲区内允许放入一件产品 */
sget = 0;                /* 缓冲区内没有产品 */
```

```
process producer {
   L1:
   produce a product;
   P(sput);
   B = product;
   V(sget);
   goto L1;
}
```

```
process consumer {
   L2:
   P(sget);
   product= B;
   V(sput);
   consume a product;
   goto L2;
}
```

1 生产者 1 消费者 N 缓冲区

```
Int B[k];                // 共享缓冲区队列
Semaphore sput;          // 可以使用的空缓冲区数
Semaphore sget;          // 缓冲区内可以使用的产品数
sput = k;                // 缓冲区内允许放入 k 件产品
sget = 0;                // 缓冲区内没有产品
Int putptr, getptr;      // 循环队列指针
putptr = 0; getptr = 0;
```

```
process producer_i {
   L1:produce a product;
   P(sput);
   B[putptr] = product;
   putptr =(putptr+1) mod k;
   V(sget);
   goto L1;
}
```

```
process consumer_j {
   L2:P(sget);
   product= B[getptr];
   getptr=(getptr+1) mod k;
   V(sput);
   consume a product;
   goto L2;
}
```

N 生产者 N 消费者 N 缓冲区

```
Int B [k];
Semaphore sput;              /* 可以使用的空缓冲区数 */
Semaphore sget;              /* 缓冲区内可以使用的产品数 */
sput = k;                    /* 缓冲区内允许放入 k 件产品 */
sget = 0;                    /* 缓冲区内没有产品 */
Int putptr, getptr; putptr = 0; getptr = 0;
s1,s2:semaphore;             /* 互斥使用 putptr,getptr */
s1 = 1; s2 = 1;
```

```
process producer_i {
    L1:produce a product;
    P(sput);
    P(s1);
    B[putptr] = product;
    putptr =(putptr+1) mod k;
    V(s1);
    V(sget);
    goto L1;
}
```

```
process consumer_j {
    L2:P(sget);
    P(s2);
    Product= B[getptr];
    getptr=(getptr+1) mod k;
    V(s2);
    V(sput);
    consume a product;
    goto L2;
}
```

苹果橘子问题

```
Int plate;
Semaphore sp;          /* 盘子里可以放几个水果 */
Semaphore sg1;         /* 盘子里有桔子 */
Semaphore sg2;         /* 盘子里有苹果 */
sp = 1;                /* 盘子里允许放入一个水果*/
sg1 = 0;               /* 盘子里没有桔子 */
sg2 = 0;               /* 盘子里没有苹果*/
```

```
process father {
        L1: 削一个苹果;
        P(sp);
        把苹果放入plate;
        V(sg2);
        goto L1;
}
process mother {
        L2: 剥一个桔子;
        P(sp);
        把桔子放入plate;
        V(sg1);
        goto L2;
}
```

```
process son {
        L3: P(sg1);
        从plate中取桔子;
        V(sp);
        吃桔子;
        goto L3;
}
process daughter {
        L4: P(sg2);
        从plate中取苹果;
        V(sp);
        吃苹果;
        goto L4;
}
```

读者写者问题

读者优先

```
semaphore rmutex，wmutex；
   rmutex=1; wmutex=1; S=1; //增加互斥信号量S
int readcount=0; //读进程计数
```

```
process reader_i( ) {
while (true) {
    P(rmutex);
      if (readcount==0) P(wmutex);
         readcount++;
    V(rmutex);
        读文件；
    P(rmutex);
         readcount--;
      if(readcount==0)  V(wmutex);
    V(rmutex);
  }
}
```

```
process writer_i( ) {
   while(true) {
      P(wmutex);
        写文件；
      V(wmutex);
    }
 }
```

？什么问题

读者优先！

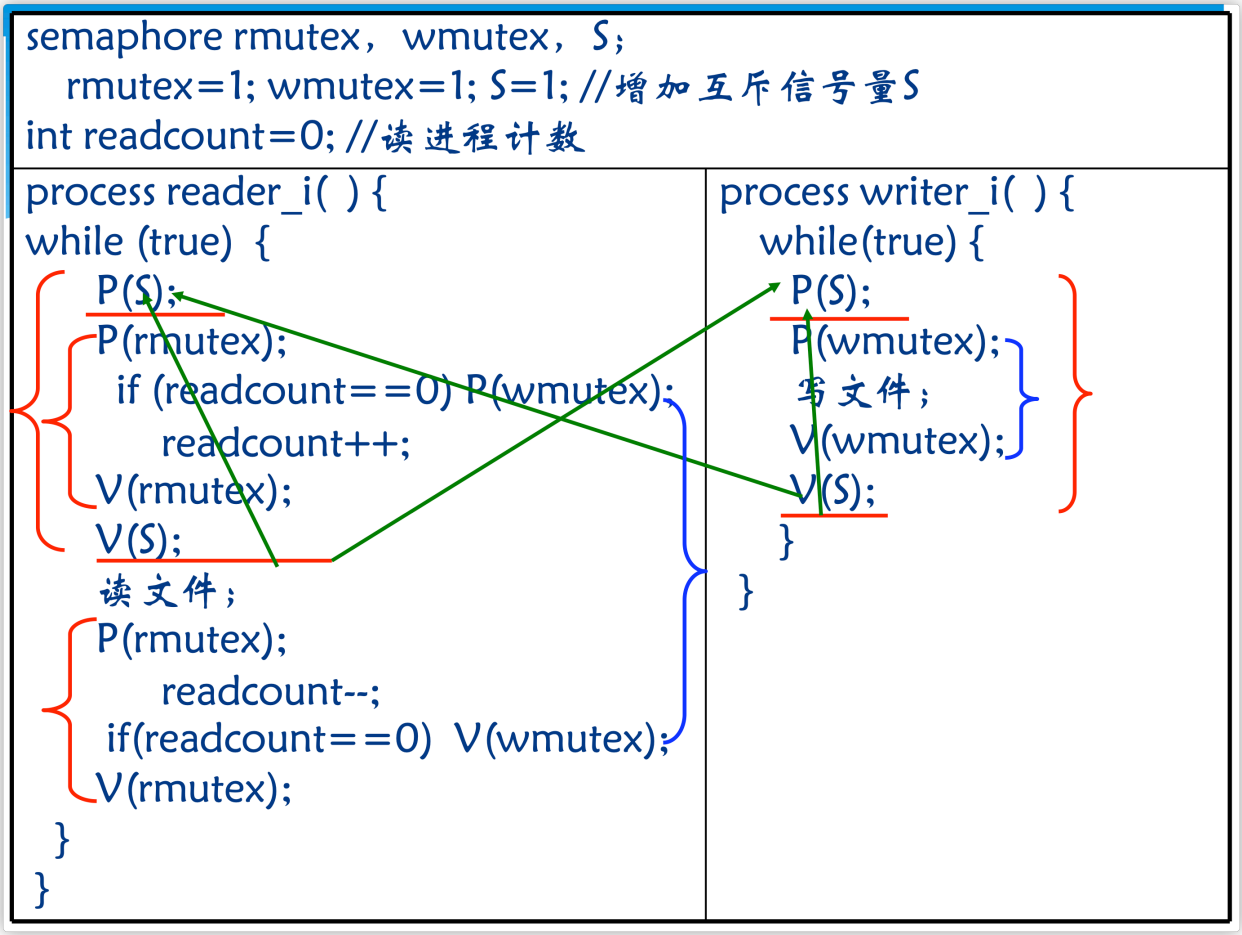
写者优先

```
int readcount = 0, writecount = 0;
semaphore x=1, y=1, z=1;        // readcount,writecount互斥
semaphore rmutex=1,wmutex=1;       // 读锁，写锁
```

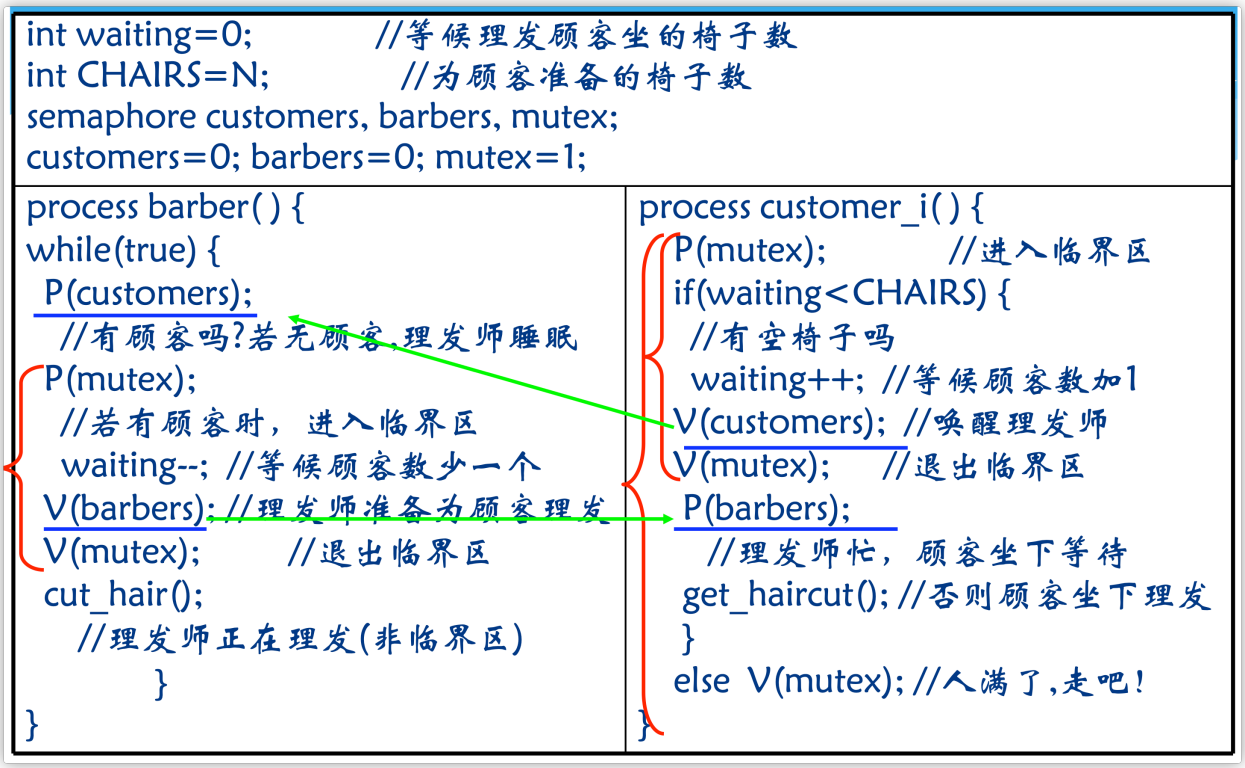
```
process reader
{
 P(z);
   P(rmutex);
    P(x);
     readcount++;
     if (readcount==1) P(wmutex);
    V(x);
  V(rmutex);
 V(z);
    read;
 P(x);
    readcount--;
    if (readcount==0) V(wmutex);
 V(x)
};
```

```
process writer
{
 P(y);
   writecount++;
   if (writecount==1) P(rmutex);
 V(y);
  P(wmutex);
    write;
 V(wmutex)
 P(y);
    writecount--;
    if (writecount==0) V(rmutex);
 V(y);
};
```

平等


semaphore rmutex, wmutex, S;
rmutex=1; wmutex=1; S=1; //增加互斥信号量S
int readcount=0; //读进程计数
process reader_i( ) {
while (true) {
P(S);
P(rmutex);
if (readcount==0) P(wmutex);
readcount++;
V(rmutex);
V(S);
读文件;
P(rmutex);
readcount--;
if(readcount==0) V(wmutex);
V(rmutex);
}
}
process writer_i( ) {
while(true) {
P(S);
P(wmutex);
写文件;
V(wmutex);
V(S);
}
}

睡眠的理发师问题


int waiting=0; //等候理发顾客坐的椅子数
int CHAIRS=N; //为顾客准备的椅子数
semaphore customers, barbers, mutex;
customers=0; barbers=0; mutex=1;
process barber( ) {
while(true) {
P(customers);
//有顾客吗?若无顾客,理发师睡眠
P(mutex);
//若有顾客时, 进入临界区
waiting--; //等候顾客数少一个
V(barbers); //理发师准备为顾客理发
V(mutex); //退出临界区
cut_hair();
//理发师正在理发(非临界区)
}
}
process customer_i( ) {
P(mutex); //进入临界区
if(waiting<CHAIRS) {
//有空椅子吗
waiting++; //等候顾客数加1
V(customers); //唤醒理发师
V(mutex); //退出临界区
P(barbers);
//理发师忙, 顾客坐下等待
get_haircut(); //否则顾客坐下理发
}
else V(mutex); //人满了,走吧!
}

农夫猎人问题

```
semaphore Scage=1;
semaphore Stiger=0;
semaphore Ssheep=0;
```

void hunter()	void peasant()	void hotel()	void zoo()
{	{	{	{
while (true) {	while (true) {	while (true) {	while (true){
……	……	P(Ssheep);	P(Stiger);
P(Scage);	P(Scage);	将羊取出笼中；	将虎取出笼中；
将虎放入笼中；	将羊放入笼中；	V(Scage);	V(Scage);
V(Stiger);	V(Ssheep);	……	……
}	}	}	}
}	}	}	}

```
void main()
{
        parbegin(hunter, peasant, hotel, zoo);
}
```

银行业务问题

```
var customer_count, server_count, mutex: semaphore;
   customer_count:=0; server_count:=n;
   mutex:=1;
```

```
process customeri(i=1,2,… .)
   begin
      take a number;
      P(mutex);
      等待区找到空沙发坐下；
      V(mutex);
      V(customer_count);
      P(server_count);
   end;
```

```
Process servers j(j=1,2,3,…)
   Begin
    L: P(customer_count);
      P(mutex);
    被呼号顾客离开沙发走出等待区；
      V(mutex);
     为该号客人服务；
     客人离开；
      V(server_count);
    go to L;
   end;
```

缓冲区管理

```
var mutex,empty,full:semaphore;
count,in:integer
buffer:array[0..79] of char;
mutex=1;empty=80;full=0;
count=0;in=0;
```

```
process Pi(i=1,...,n))
begin
 L: 读入一字符到x;
 P(empty);
 P(mutex);
  Buffer[in]=x;
  in=(in+1) % 80;
  count++;
  if (count==80)
   {count=0; V(mutex); V(full); }
  else V(mutex);
 goto L;
end;
```

```
process Q
begin
 while(true) {
 P(full);
 P(mutex);
  for(int j=0; j< 80;j++)
  read buffer[j];
  in:=0;
 V(mutex);
  for (int j=0; j< 80;j++)
 V(empty);
  }
end;
```

售票问题

```
Var run1, run2, stop1, stop2: semaphore;
run1:=0; run2:=0; stop1:=0; stop2:=0;
```

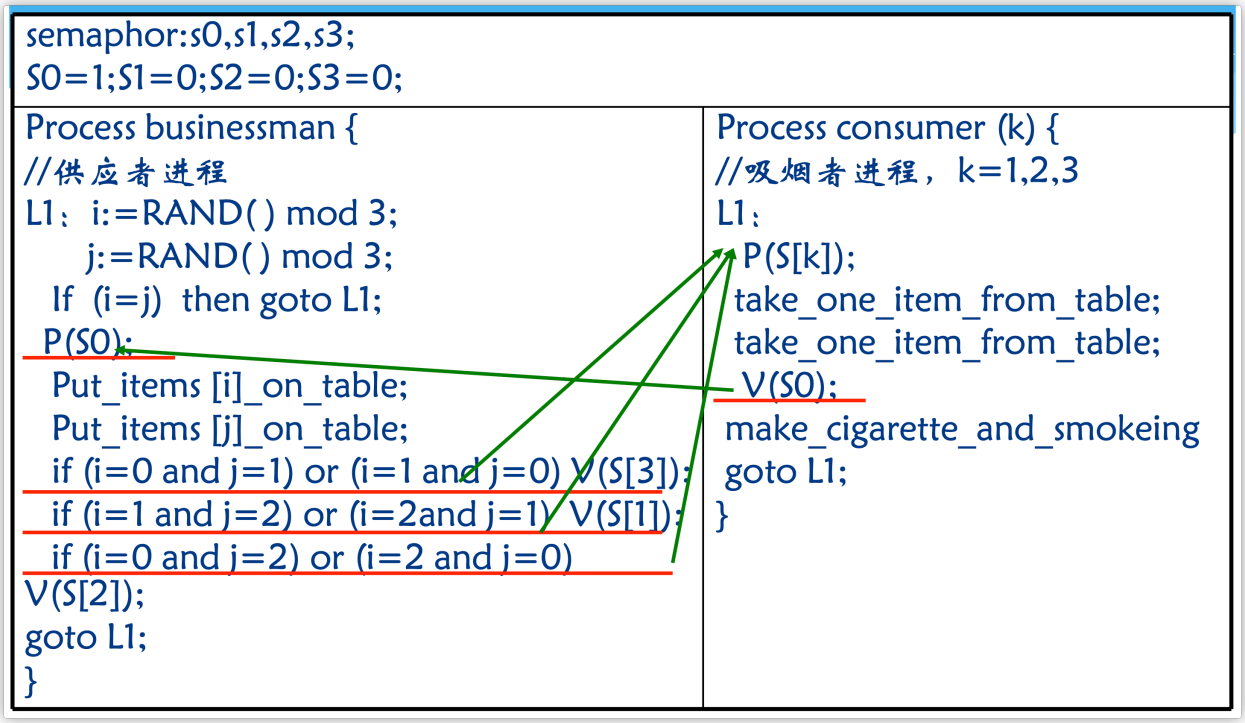
```
void Driver() {
  while (true)
  {
     P(run1);
     P(run2);
     开车;
     停车;
     V(stop1);
     V(stop2);
   }
}
```

```
void Seller1() {
  while (true)  {
     上乘客;
     关车门;
     V(run1);
     售车票;
    P(stop1);
     开车门;
     下乘客;
   }
}
```

```
void Seller2() {
  while (true)    {
     上乘客;
     关车门;
     V(run2);
     售车票;
    P(stop2);
     开车门;
     下乘客;
   }
}
```

```
void main() {
   parbegin(Driver; Seller1; Seller2);
}
```

吸烟者问题



哲学家就餐问题

有死锁

```
semaphore fork[5];
for (int i=0;i<5;i++)
fork[i]=1;
cobegin
process philosopher_i( ) {  //i= 0,1,2,3,4
  while(true) {
    think( );
    P(fork[i]);                    //先取右手的叉子
    P(fork[(i+1)%5]);              //再取左手的叉子
       eat( );
    V(fork[i]);
    V(fork[(i+1)%5]);
     }
}
coend
```

存在什么问题？

死锁！

避免死锁方法

- 至多允许四个哲学家同时取叉子 (C. A. R. Hoare方案)
- 奇数号先取左手边的叉子，偶数号先取右手边的叉子
- 每个哲学家取到手边的两把叉子才吃，否则一把叉子也不取 (第五版教材, Page 188, AND型信号量)

控制人数

```
  semaphore fork[5];
   for (int i=0;i<5;i++)
         fork[i]= 1;
   semaphore room=4; //增加一个侍者，设想有两个房间1号房间是会议室，2号房间是餐厅
cobegin
process philosopher_i( ){/*i=0,1,2,3, 4 */
          while(true) {
          think( );
          P(room); //控制最多允许4位哲学家进入2号房间餐厅取叉子
          P(fork[i];
          P(fork[(i+1)%5] ) ;
            eat( );
          V(fork[i]);
          V(fork([i+ 1] % 5);
          V(room);
    }
 }
coend
```

分左右手

```
void philosopher (int i)
{  if  i  mod 2＝＝0 then
   {
          P(fork[i]);          //偶数哲学家先右手
          P(fork[(i+1) mod 5 ]); //后左手
                eat();
          V(fork[i]);
          V (fork[(i+1) mod 5]);
   }
   else
    {
          P (fork[(i+1) mod 5 ]); //奇数哲学家，先左手
          P (fork[i]);  //后右手
                eat();
          V(fork[(i+1) mod 5]);
          V(fork[i]);
   }
}
```


独木桥问题

一方全部通行

```
var wait,mutex1,mutex2:semaphore;
mutex1=mutex2=1;wait=1;
counter1,counter2:integer;
```

```
process P东(){
  while(true){
    P(mutex1);
      count1++;
      if (count1==1) P(wait);
    V(mutex1);
      过独木桥;
    P(mutex1);
      count1--;
      if (count1==0) V(wait);
    V(mutex1);
  }
}
```

```
process P西(){
  while(true){
    P(mutex2);
      count2++;
      if (count2==1) P(wait);
    V(mutex2);
      过独木桥;
    P(mutex1);
      count2--;
      if (count2==0) V(wait);
    V(mutex2);
  }
}
```

桥上最多 K 辆车

```
semaphore wait,mutex1,mutex2,bridge;
mutex1=mutex2=1;bridge=k;wait=1;
int count1,count2; count1=0;count2=0;
```

```
process P东( ){
  P(mutex1);
    count1++;
    if (count1==1)  P(wait);
  V(mutex1);
  P(bridge);
  {过桥};
  V(bridge);
  P(mutex1);
    count1--;
    if (count1==0)  V(wait);
  V(mutex1);
 }
```

```
process P西( ){
  P(mutex2);
    count2++;
    if (count2==1)  P(wait);
  V(mutex2);
  P(bridge);
  {过桥};
  V(bridge);
  P(mutex2);
    count2--;
    if (count2==0)  V(wait);
  V(mutex2);
 }
```

每三辆车交替

```
semaphore wait,mutex1,mutex2;
mutex1=mutex2=1;wait=1;
int counter1,counter2; counteru1=0; countd1=0; counteru2=0; counterd2=0;
semaphore S1,S2;S1=3;S2=0;
```

```
Process P东() {
while(true) {
  P(S1)
    P(mutex1);
      countu1++;
      if (countu1==1)&&(countd1==0)
                          P(wait);
    V(mutex1);
      过独木桥;
  V(S2)
    P(mutex1);
    countu1--;    countd1++;
    if ((countu1==0)&&(countd1==3))
     {countd1=0;  V(wait); }
   V(mutex1);
   }
}
```

```
Process P西() {
while(true) {
  P(S2)
    P(mutex2);
      countu2++;
      if (countu2==1)&&(countd2==0)
                          P(wait);
    V(mutex2);
      过独木桥;
  V(S1)
    P(mutex2);
     countu2--;  countd2++;
    if ((countu2==0)&&(countd2==3))
     {countd2=0;   V(wait); }
    V(mutex2);
   }
}
```

22:55

串行

```
semaphore stop,wait,mutex1,mutex2;
stop=mutex1=mutex2=1;wait=1;
int count1,count2; count1=0;count2=0;
```

```
process P东( ) {
 P(stop);
   P(mutex1);
     count1++;
     if (count1==1)  P(wait);
    V(mutex1);
 V(stop);
    {过桥};
   P(mutex1);
     Count1--;
    if (count1==0)  V(wait);
   V(mutex1);
  }
```

```
process P西( ) {
  P(stop);
    P(mutex2);
     count2++;
     if (count2==1)  P(wait);
    V(mutex2);
  V(stop);
    {过桥};
   P(mutex2);
     count2--;
     if (count2==0)  V(wait);
   V(mutex2);
     }
```

22:55

管程

实现

```
typedef struct InterfaceModule { //InterfaceModule是结构体的名字
        semaphore mutex;        //进程调用管程过程前使用的互斥信号量
        semaphore next;         //发出signal的进程挂起自己的信号量
        int next_count; };      //在next上等待的进程数
    mutex=1;next=0;next_count=0; //初始化语句
```

```
void enter(InterfaceModule &IM) {
   P(IM.mutex); //判有否发出过signal的进程?
}
```

```
void leave(InterfaceModule &IM) {
   if (IM.next_count>0)
        V(IM.next);
     //有就释放一个发出过signal的进程
  else V(IM.mutex); } //否则开放管程
```

```
void wait(semaphore &x_sem, int &x_count,InterfaceModule &IM) {
 x_count++; //等资源进程个数加1,
 if (IM.next_count>0)
      //判断是否有发出过signal的进程
    V(IM.next); //有就释放一个
 else
    V(IM.mutex); //否则开放管程
 P(x_sem); //等资源进程阻塞自己,
           //x_sem初始化为0
 x_count--; } //等资源进程个数减1
```

```
void signal(semaphore &x_sem,int &x_count,InterfaceModule &IM)
{ if(x_count>0) { //判断等待进程
  IM.next_count++;
        //发出signal进程个数加1
  V(x_sem);  //释放一个等资源的进程
  P(IM.next); //发出signal进程阻塞自己
  IM.next_count--; }
      //发出signal进程个数减1
}
```

读者写者问题

```
TYPE  read-write=monitor
   int rc,wc;
     semaphore R,W;R=0;W=0;
     int R_count,W_count;
     rc=0; wc=0;
InterfaceModule IM;
DEFINE start_read, end_read, start_write, end_write;
USE  wait,signal,enter,leave;
```

```
void start_read( ){
  enter(IM);
   if(wc>0) wait(R,R_count,IM);
                    rc++;
          signal(R, R_count, R IM);
  leave(IM);   }
```

```
void start_write( ) {
  enter(IM);
      wc++;
      if(rc>0 || wc>1)  wait(W,W_count,IM);
  leave(IM);
}
```

```
void end_read( ) {
  enter(IM);
      rc--;
      if(rc==0)
         signal(W,W_count,IM);
  leave(IM);  }
```

```
void end_write( ) {
  enter(IM);
      wc-- ;
      if(wc>0) signal(W,W_count,IM);
      else signal(R, R_count, IM);
  leave(IM);    }
```

```
process P1( ) {
  ……
   read-write.start_read( );
    {read};
   read-write.end_read( );
  …… }
```

```
process P2( ) {
  ……
   read-write.start_write( );
    {write};
   rear-write.end_write( );
  …… }
```

哲学家就餐问题

```
type dining_philosophers=monitor
  enum {thinking, hungry, eating} state[5];
  semaphore self[5];   int self_count[5];   InterfaceModule IM;
  for (int i=0;i<5;i++)  state[i]=thinking; //初始化，i为进程号
 define pickup, putdown;
 use enter, leave, wait, signal
```

```
void pickup(int i) { //i=0,1,...,4
    enter(IM);
        state[i]=hungry;
        test(i);
        if(state[i]!=eating)
            wait(self[i],self_count[i],IM);
    leave(IM);
}
```

```
void putdown(int i)
{    //i=0,1,2,..,4
    enter(IM);
        state[i]=thinking;
        test((i-1)%5);
        test((i+1)%5);
    leave(IM);
}
```

```
void test(int k)  {      //k=0,1,...,4
    if((state[(k-1)%5]!=eating)&&(state[k]==hungry)
          &&(state[(k+1)%5]!=eating)) {
             state[k]=eating; signal(self[k],self_count[k],IM);
          }
   }
}
```

生产者消费者问题

```
type producer_consumer=monitor
item B[k]; int in, out; //B[k]表示缓冲单元， in, out是存取指针
int count;    //缓冲中产品数
semaphore notfull, notempty; //条件变量
int notfull_count, notempty_count; InterfaceModule IM;
define append,take;
use enter,leave,wait,signal;
```

```
void append(item x) {
  enter(IM);
    if(count==k)   //缓冲已满
  wait(notfull,notfull_count,IM);
      B[in]=x;
      in=(in+1)%k;
      count++; //增加一个产品
signal(notempty,notempty_count,IM);
            //唤醒等待消费者
  leave(IM);
}
```

```
void take(item &x) {
   enter(IM);
     if(count==0) //缓冲已空
wait(notempty,notempty_count,IM);
        x=B[out];
        out=(out+1)%k;
        count--;      //减少一个产品
   signal(notfull,notfull_count,IM);
         //唤醒等待生产者
   leave(IM);
}
```

```
process producer_i( ) { //i=1,…,n
    item x;
          produce(x);
    producer_consumer.append(x)
}
```

```
process consumer_j( ) { //j=1,…m
          item x;
    producer_consumer.take(x);
          consume(x);
}
```

苹果橘子问题

```
type FMSD=MONITOR
enum FRUIT {apple,orange} plate;  bool full;
semaphore SP, SS, SD;  int SP_count, SS_count, SD_count;
full=false; InterfaceModule IM; DEFINE put,get;
USE enter,leave,wait,signal;
```

```
void put(FRUIT fruit) { // fruit:
apple or orange
   enter(IM);
     if(full)
       wait(SP,SP_count,IM);
         full=true;
         plate=fruit;
     if(fruit==orange)
       signal(SS,SS_count,IM);
     else   signal(SD,SD_count,IM);
   leave(IM); }
```

```
void get(FRUIT fruit, FRUIT &x) {
    enter(IM);
      if (!full ‖ plate != fruit) {
        if (fruit==orange)
            wait(SS,SS_count,IM);
        else wait(SD,SD_count,IM);
        }
        x=plate;
         full=false;
        signal(SP,SP_count,IM);
      leave(IM);          }
```

```
process father( ){
    {准备好苹果};
    FMSD.put(apple);      }
```

```
process son( ){
     FMSD.get(orange, x);
     {吃取到的桔子};          }
```

```
process mother( ){
    {准备好桔子};
    FMSD.put(orange);    }
```

```
process daughter( ){
      FMSD.get(apple, x);
      {吃取到的苹果};          }
```